

TECHNICAL DOCUMENT

ECOM24

E-Commerce Transaction Processing System

Technical Operations Manual

Architectural and Operational Specification

Document Code: DOC-ECOM24-001

Version: 1.0

Date: February 08, 2026

Classification: INTERNAL - COMPANY USE ONLY

Status: APPROVED

KNOWLEDGE TRANSFER DOCUMENT

Revision History

Version	Date	Author	Change Description
1.0	02/08/2026	Engineering Team	Initial document release

Approval Register

Role	Name	Date	Signature
Prepared by			
Reviewed by			
Approved by			

Table of Contents

1. System Introduction

1.1 Document Purpose

This document constitutes the complete technical operational specification for the ECOM24 (E-Commerce 24-Task) system. Its primary purpose is to provide a detailed description of the software architecture, configuration parameters, temporal dependencies, and operational dynamics of the system, in order to support development, maintenance, testing, and operational management activities.

This document has been prepared with particular attention to knowledge transfer, thus representing a fundamental resource for onboarding new technical personnel and ensuring operational continuity of the system.

1.2 Scope of Application

The ECOM24 system represents a complex software architecture specifically designed for the management and orchestration of a high-throughput e-commerce transaction processing pipeline. The system was conceived to operate in a high-demand business environment where reliability, timing precision, and efficient use of computational resources are fundamental requirements for business success.

The system architecture is based on a distributed execution model that coordinates multiple specialized services, each responsible for specific functional domains of the transaction lifecycle. These services orchestrate the execution of a large set of interdependent tasks that must comply with strict timing constraints and well-defined precedence relationships.

1.3 Intended Audience

This document is intended for: software engineers responsible for system development and maintenance, system operators responsible for operational monitoring and control, quality assurance teams for verification and validation activities, training personnel for knowledge transfer programs, and technical management for project activity supervision.

2. Global Configuration and Operational Parameters

2.1 Operational Time Window

The ECOM24 system operates within a well-defined time window extending from the initial instant, identified as time zero, up to time unit three hundred and ninety. This extensive operational window of three hundred and ninety time units represents the period during which all scheduled operations must be started, executed, and completed.

The choice of this specific duration is closely linked to the complexity of the business processes and the need to accommodate a large number of sequential and parallel tasks, ensuring that all critical operations, including repetitive and delayed executions, can be completed within the defined processing cycle.

2.2 System Parameters

Parameter	Value	Description
System Name	24tasks	Unique instance identifier
Operational Window (t_max)	390 t.u.	Maximum operational cycle duration
Max Resources (r_max)	190	Simultaneous processing units
Active Services	8	Number of service modules
Total Tasks	24	Number of operational tasks

The availability of one hundred and ninety resources allows a very high degree of parallelism in task execution, enabling the system to effectively manage high workloads and complex processing pipelines. This substantial resource pool is essential for orchestrating the 24 distinct operational tasks, many of which have their own concurrency limits and resource requirements.

3. Modular Service Architecture

3.1 Architecture Overview

The ECOM24 system architecture is structured into eight main services, each designed to manage a specific functional domain of the e-commerce transaction lifecycle. This modular organization enables a clear separation of responsibilities and facilitates both maintenance and system evolution over time.

ID	Service	Primary Responsibility
1	Payment_Service	Manages all financial aspects of the transaction, including validation, tax, risk, and fraud detection
2	Inventory_Service	Handles stock checking, warehouse allocation, and quality control
3	Order_Service	Orchestrates core order processing, confirmation, documentation, and archival
4	Shipping_Service	Manages logistics calculations related to order fulfillment
5	Notification_Service	Responsible for all customer-facing communications
6	Security_Service	Ensures data security, compliance, and auditing throughout the process
7	Analytics_Service	Performs performance monitoring and business intelligence reporting
8	Infrastructure_Service	Manages underlying system operations like backups and cleanup

3.2 Detailed Service Descriptions

3.2.1 Payment_Service (ID: 1)

The Payment_Service is the financial core of the system. It is responsible for a wide range of critical tasks including initial Payment_Validation, Tax_Calculation, Risk_Assessment, Fraud_Detection, Currency_Conversion, and Price_Optimization. This service ensures the financial integrity and security of every transaction.

3.2.2 Inventory_Service (ID: 2)

The Inventory_Service manages the physical or digital stock. It performs the initial Inventory_Check to ensure product availability, handles the subsequent Warehouse_Allocation for fulfillment, and runs a final Quality_Check. Its correct operation is crucial for preventing stockouts and ensuring product quality.

3.2.3 Order_Service (ID: 3)

The Order_Service acts as a central hub for the transaction lifecycle. It manages Order_Confirmation, Document_Generation for invoices or receipts, Final_Processing steps, long-term Archive_Processing, and a Final_Validation to close the order. This service orchestrates many key dependencies within the system.

3.2.4 Shipping_Service (ID: 4)

The Shipping_Service is a specialized module focused exclusively on logistics. Its single task, Shipping_Calculation, determines shipping costs and methods. Despite being a single-task service, it is a critical input for the final order cost and is a dependency for multiple downstream processes.

3.2.5 Notification_Service (ID: 5)

The Notification_Service is responsible for all communications with the end customer. Its task, Customer_Notification, ensures that users are kept informed about the status of their orders. This service is a key component for customer satisfaction and experience.

3.2.6 Security_Service (ID: 6)

The Security_Service is dedicated to maintaining the integrity and security of the system and its data. It performs Compliance_Checks, Security_Validation, Data_Encryption, and maintains a detailed Audit_Trail of all significant operations, ensuring the system adheres to security standards and regulations.

3.2.7 Analytics_Service (ID: 7)

The Analytics_Service provides business intelligence and operational monitoring capabilities. It includes the Performance_Monitor task for real-time system health checks and the Report_Generation task for creating business-level summaries and insights from the processed transactions.

3.2.8 Infrastructure_Service (ID: 8)

The Infrastructure_Service handles essential background operations that support the entire system. It is responsible for periodic Backup_Creation to ensure data durability and for running the System_Cleanup task to maintain system hygiene and performance over time.

4. Detailed Description of Operational Tasks

4.1 Complete Task Registry

ID	Task Name	Task Code	Service ID	Duration	Resources	Max Concur.	Rep
1	Payment_Validation	PAY_VAL	1	5	4	4	0/0
2	Inventory_Check	INV_CHK	2	4	5	5	3/0
3	Shipping_Calculation	SHIP_CALC	4	10	3	3	3/0
4	Order_Confirmation	ORD_CONF	3	8	2	2	0/0
5	Customer_Notification	CUST_NOTIF	5	10	1	4	0/9
6	Warehouse_Allocation	WH_ALLOC	2	10	4	1	1/0
7	Quality_Check	QC_CHK	2	7	4	4	0/0
8	Final_Processing	FINAL_PROC	3	10	1	2	3/1
9	Tax_Calculation	TAX_CALC	1	5	5	2	1/0
10	Document_Generation	DOC_GEN	3	4	5	5	2/7
11	Compliance_Check	COMP_CHK	6	9	4	5	2/0
12	Archive_Processing	ARCH_PROC	3	9	3	5	3/1
13	Security_Validation	SEC_VAL	6	4	2	1	0/0
14	Performance_Monitor	PERF_MON	7	5	2	2	0/2
15	Backup_Creation	BCK_CREA	8	3	4	3	2/15
16	Report_Generation	RPT_GEN	7	3	2	4	1/9
17	Data_Encryption	DATA_ENCR	6	4	2	4	1/6
18	Audit_Trail	AUD_TRL	6	4	5	3	0/4
19	System_Cleanup	SYS_CLN	8	5	1	3	0/0
20	Final_Validation	FINAL_VAL	3	7	4	3	0/0
21	Risk_Assessment	RISK_ASMT	1	9	3	4	3/15
22	Fraud_Detection	FRD_DET	1	4	5	3	0/0
23	Currency_Conversion	CUR_CONV	1	7	1	5	0/0

ID	Task Name	Task Code	Service ID	Duration	Resources	Max Concur.	Rep
24	Price_Optimization	PRC_OPT	1	3	3	2	3/0

Legend: Duration = time units; Resources = required computational units; Max Concur. = maximum concurrent executions; Rep = repetitions (rc/rd format where rc = repeat count, rd = repeat delay)

A notable characteristic of this system is the heterogeneous nature of its tasks. Resource requirements vary from 1 to 5 units, and durations span from 3 to 10 time units. Many tasks are configured for repetition, some with significant delays, reflecting complex, recurring business logic within the operational cycle.

5. Temporal Constraints and Dependency System

5.1 Start-to-Start Precedence Model

The ECOM24 system implements a sophisticated temporal constraint system based on the concept of start-to-start precedence. This type of constraint specifies that a destination task can begin execution only after a given delay has elapsed since the start of the source task execution.

This system utilizes a critical flag, `wait_all`, to control the behavior of dependencies involving repetitive tasks:

- * `wait_all: false`: The destination task can begin after the specified delay has elapsed from the *first* execution of the source task. This enables maximum parallelism, allowing downstream processes to start without waiting for all repetitions of the antecedent task to complete.
- * `wait_all: true`: The destination task must wait until *all scheduled repetitions* of the source task have finished. The delay then applies from the start time of the *last* repetition. This is used to enforce strict synchronization points in the workflow, ensuring that a process only begins after its prerequisite has fully completed its entire cycle.

5.2 Dependency Matrix

Source Task	Destination Task	Delay (t.u.)	Type	Wait All
Payment_Validation (1)	Inventory_Check (2)	5	SS	false
Inventory_Check (2)	Shipping_Calculation (3)	9	SS	true
Payment_Validation (1)	Order_Confirmation (4)	21	SS	false
Inventory_Check (2)	Order_Confirmation (4)	16	SS	true
Shipping_Calculation (3)	Order_Confirmation (4)	10	SS	true
Order_Confirmation (4)	Customer_Notification (5)	7	SS	false
Inventory_Check (2)	Customer_Notification (5)	15	SS	false
Order_Confirmation (4)	Warehouse_Allocation (6)	10	SS	false
Shipping_Calculation (3)	Quality_Check (7)	7	SS	false
Payment_Validation (1)	Quality_Check (7)	11	SS	false
Warehouse_Allocation (6)	Final_Processing (8)	13	SS	true
Order_Confirmation (4)	Final_Processing (8)	12	SS	true
Customer_Notification (5)	Tax_Calculation (9)	6	SS	false
Shipping_Calculation (3)	Tax_Calculation (9)	13	SS	true
Tax_Calculation (9)	Document_Generation (10)	5	SS	true
Final_Processing (8)	Document_Generation (10)	13	SS	false
Order_Confirmation (4)	Document_Generation (10)	14	SS	false
Final_Processing (8)	Compliance_Check (11)	9	SS	true
Payment_Validation (1)	Compliance_Check (11)	9	SS	false
Order_Confirmation (4)	Compliance_Check (11)	16	SS	false
Payment_Validation (1)	Archive_Processing (12)	15	SS	true
Shipping_Calculation (3)	Security_Validation (13)	5	SS	false
Customer_Notification (5)	Security_Validation (13)	6	SS	true
Tax_Calculation (9)	Security_Validation (13)	6	SS	false
Compliance_Check (11)	Performance_Monitor (14)	7	SS	false
Customer_Notification (5)	Backup_Creation (15)	9	SS	false
Tax_Calculation (9)	Backup_Creation (15)	12	SS	false

Source Task	Destination Task	Delay (t.u.)	Type	Wait All
Archive_Processing (12)	Backup_Creation (15)	8	SS	false
Shipping_Calculation (3)	Report_Generation (16)	14	SS	true
Quality_Check (7)	Report_Generation (16)	6	SS	false
Order_Confirmation (4)	Report_Generation (16)	11	SS	false
Tax_Calculation (9)	Data_Encryption (17)	9	SS	false
Performance_Monitor (14)	Data_Encryption (17)	14	SS	false
Payment_Validation (1)	Data_Encryption (17)	7	SS	false
Report_Generation (16)	Audit_Trail (18)	13	SS	false
Archive_Processing (12)	Audit_Trail (18)	12	SS	false
Warehouse_Allocation (6)	Audit_Trail (18)	8	SS	true
Security_Validation (13)	System_Cleanup (19)	5	SS	false
Final_Processing (8)	System_Cleanup (19)	16	SS	false
Performance_Monitor (14)	System_Cleanup (19)	16	SS	false
Customer_Notification (5)	Final_Validation (20)	5	SS	true
Report_Generation (16)	Final_Validation (20)	13	SS	false
Backup_Creation (15)	Risk_Assessment (21)	6	SS	false
Customer_Notification (5)	Fraud_Detection (22)	11	SS	true
Report_Generation (16)	Currency_Conversion (23)	9	SS	false
Currency_Conversion (23)	Price_Optimization (24)	6	SS	false
Security_Validation (13)	Price_Optimization (24)	15	SS	false
Risk_Assessment (21)	Price_Optimization (24)	10	SS	false

Legend: SS = Start-to-Start

5.3 Critical Dependency Analysis

The dependency graph of the ECOM24 system is highly interconnected, reflecting a complex business process. The initial task, **Payment_Validation** (1), serves as a primary trigger for multiple parallel execution paths, including inventory, quality control, and security checks.

Order_Confirmation (4) acts as a major synchronization point and distribution hub. It depends on the completion of several repetitive prerequisite tasks

(`Inventory_Check`, `Shipping_Calculation`), enforced by `wait_all: true` constraints. Once it executes, it enables a wide array of subsequent processes, including notifications, allocations, and documentation.

The use of `wait_all` is strategic. For instance, `Final_Processing` (8) must wait for `Warehouse_Allocation` (6) to fully complete (`wait_all: true`), ensuring physical allocation is finished before finalization begins. In contrast, `Data_Encryption` (17) can start shortly after `Payment_Validation` (1) begins (`wait_all: false`), allowing security operations to run in parallel with the main business flow.

The final tasks in the payment service, `Currency_Conversion` (23) and `Price_Optimization` (24), form a tightly coupled chain, indicating a final financial processing step that depends on earlier security and risk assessment activities.

6. Final Considerations and Recommendations

6.1 Architecture Summary

The ECOM24 system represents a sophisticated, high-concurrency architecture for managing complex, multi-stage business transactions. Its design reflects a deep understanding of operational requirements, data integrity, and the need for both parallel processing and strict sequential synchronization.

The balance between parallelism (enabled by `wait_all: false`) and synchronization points (enforced by `wait_all: true`), combined with a large pool of computational resources, demonstrates a mature and robust approach to designing mission-critical business systems. The heterogeneity of task parameters highlights the system's ability to orchestrate a diverse set of computational workloads.

6.2 Modular Architecture Benefits

The modularity of the architecture, with its clear separation into specialized services, facilitates not only system understanding but also future maintenance and evolution. The ability to modify or extend individual services (e.g., adding a new payment provider in `Payment_Service`) without impacting the entire architecture is a significant advantage for adapting the system to changing business requirements.

The strategic use of the `wait_all` flag provides granular control over the operational flow, allowing system designers to optimize for either throughput and responsiveness or for process integrity and synchronization, depending on the specific business need at each stage of the pipeline.

6.3 Knowledge Transfer Recommendations

For effective knowledge transfer, it is recommended to follow a structured training path that includes: understanding global parameters and the operational window, studying the service architecture starting with the core `Payment_Service`, `Inventory_Service`, and `Order_Service`, detailed analysis of individual tasks and their heterogeneous characteristics, mapping temporal dependencies with a strong focus on the differing semantics of `wait_all: true` and `wait_all: false`, simulation of the operational flow with particular attention to synchronization points like `Order_Confirmation`, understanding the impact of repetitive tasks with long delays (e.g., `Backup_Creation`, `Risk_Assessment`), and finally studying the roles of the `Security_Service` and `Infrastructure_Service` in maintaining system health and integrity.

Appendix A: Glossary

Term	Definition
BPM	Business Process Management - The discipline of modeling, automating, and optimizing business processes
t.u.	Time Unit - Base unit for time measurement in the system
r_max	Maximum number of computational resources allocable simultaneously
t_max	Maximum duration of operational window in time units
Start-to-Start (SS)	Type of precedence constraint based on task start rather than completion
wait_all	Constraint flag. If <code>false</code> , the dependent task can start after a delay from the source's first execution. If <code>true</code> , it must wait for all source repetitions to complete.
Pipeline	A processing sequence where the output of one phase becomes the input of the next
Periodic Task	Task configured to execute multiple repetitions at regular intervals
rc	Repeat Count - Number of repetitions for a repetitive task

Term	Definition
rd	Repeat Delay - Time delay between repetitions of a repetitive task
max_c	Maximum Concurrency - Maximum number of concurrent executions allowed for a task

Appendix B: Reference Documents

Code	Document Title	Version
REF-001	ECOM24 System Requirements Specification	[To be filled]
REF-002	Interface Control Document - Payment Gateway	[To be filled]
REF-003	Verification and Validation Plan	[To be filled]
REF-004	System Operations Manual	[To be filled]

END OF DOCUMENT